🐼 Pandas Study Notes

Python Data Analysis Library — Your Data Cleaning & Exploration Tool

For 10th Standard Students & ML Beginners

1. What is Pandas?

Pandas lets you work with data the same way you'd use an Excel spreadsheet — but with code. You can load CSV files, clean messy data, filter rows, create new columns, group data, and much more.

💡 Think of it like:	Excel + Python = Pandas. A table has rows and columns. Pandas calls this a DataFrame.

When to use Pandas:

- Loading datasets (CSV, Excel, JSON, SQL, etc.)
- Cleaning data — removing nulls, fixing types, renaming columns
- Exploring data — seeing statistics, distributions, patterns
- Preparing data before feeding into ML models

```
pip install pandas
import pandas as pd
import numpy as np   # pandas works closely with numpy
```

2. Core Data Structures

2.1 Series — A Single Column of Data

A Series is like a one-column table. It has values and an index (row labels).

```
s = pd.Series([10, 20, 30, 40])
# Index: 0  1  2  3
# Values: 10 20 30 40
```

```
# Custom index
s = pd.Series([85, 92, 78], index=['Maths', 'Science', 'English'])
s['Maths']    # 85
s[s > 80]     # Maths: 85, Science: 92
```

2.2 DataFrame — The Main Data Structure

A DataFrame is a full table with rows and columns. This is what you'll use 95% of the time.

```
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie'],
    'Marks':  [85, 72, 91],
```

```
    'Grade':  ['A', 'B', 'A+']
})
```

Name	Marks	Grade
Alice	85	A
Bob	72	B
Charlie	91	A+

3. Loading & Saving Data

Format	Load Code	Save Code
CSV file	`pd.read_csv('file.csv')`	df.to_csv('file.csv', index=False)
Excel file	`pd.read_excel('file.xlsx')`	df.to_excel('file.xlsx')
JSON file	`pd.read_json('file.json')`	df.to_json('file.json')
SQL database	`pd.read_sql(query, conn)`	df.to_sql('table', conn)
From URL	`pd.read_csv('https://...')`	—
Parquet	`pd.read_parquet('file.parquet')`	df.to_parquet('file.parquet')

```
# Useful options when loading CSV
df = pd.read_csv('data.csv',
    sep=',',           # separator character
    header=0,          # which row is the header
    index_col='ID',    # use this column as row index
    nrows=100,         # only load first 100 rows
    na_values=['?'])   # treat '?' as NaN
```

4. Exploring Your Data — First Steps Always

When you get a new dataset, always start with these commands:

Command	What It Shows
df.head(5)	`First 5 rows (default 5, change the number)`
df.tail(5)	`Last 5 rows`
df.shape	`(rows, columns) — how big is the data?`
df.info()	`Column names, data types, non-null count`
df.describe()	`Stats: mean, std, min, max for number columns`
df.columns	`List of all column names`
df.dtypes	`Data type of each column`

df.nunique()	Number of unique values per column
df.value_counts()	Count of each unique value in a Series
df.isnull().sum()	Count of missing values per column
df.sample(5)	Random 5 rows — great for quick check
df.memory_usage()	How much RAM the data uses

```
# Quick full overview workflow
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

5. Selecting Data — Rows & Columns

5.1 Selecting Columns

```
df['Name']           # single column → returns a Series
df[['Name', 'Marks']]  # multiple columns → returns DataFrame
```

5.2 Selecting Rows — loc vs iloc

loc = select by LABEL (name). iloc = select by POSITION (number).

```
df.loc[0]            # row with index label 0
df.loc[0:2]          # rows 0, 1, 2 (INCLUSIVE of end!)
df.loc[0, 'Name']    # row 0, column 'Name'
df.loc[:, 'Marks']   # all rows, 'Marks' column
```

```
df.iloc[0]           # first row (by position)
df.iloc[0:2]         # rows 0, 1 (EXCLUSIVE of end, like Python)
df.iloc[0, 1]        # row 0, column at position 1
df.iloc[:, 1:3]      # all rows, columns 1 and 2
```

> **i** loc uses labels (like 'Name'), iloc uses numbers (position 0, 1, 2). When in doubt, use iloc for position-based access.

5.3 Filtering Rows by Condition

```
df[df['Marks'] > 80]                    # rows where Marks > 80
df[df['Grade'] == 'A']                  # exact match
df[(df['Marks'] > 70) & (df['Grade'] == 'A')]  # AND condition
df[(df['Marks'] < 60) | (df['Marks'] > 90)]    # OR condition
df[df['Name'].isin(['Alice', 'Bob'])]   # name is in list
df[~df['Name'].isin(['Alice'])]          # NOT in list
```

```
df[df['Name'].str.startswith('A')]        # string filter
```

5.4 Using query() — Readable Filters

```
df.query('Marks > 80')
df.query('Marks > 80 and Grade == "A"')
```

6. Handling Missing Data — Critical for ML

Real datasets are ALWAYS messy. Missing values (NaN = Not a Number) break ML models. Here's how to handle them:

6.1 Detect Missing Values

```
df.isnull()           # True/False for each cell
df.isnull().sum()     # count of NaN per column
df.isnull().mean()    # % of missing per column
df.notnull()          # opposite — where data EXISTS
```

6.2 Remove Missing Values

```
df.dropna()                # drop ANY row with NaN
df.dropna(axis=1)          # drop columns with NaN
df.dropna(subset=['Marks'])  # only drop if 'Marks' is NaN
df.dropna(thresh=3)         # keep rows with at least 3 non-NaN
```

6.3 Fill Missing Values

```
df.fillna(0)                        # fill all NaN with 0
df['Marks'].fillna(df['Marks'].mean())  # fill with average
df['Grade'].fillna('Unknown')       # fill with a default string
df.fillna(method='ffill')           # fill with previous value
df.fillna(method='bfill')           # fill with next value
```

🎯 Best Practice:	For numbers, fill with median (not mean) — median is not affected by extreme values. For categories, fill with mode (most common value).

```
df['Age'].fillna(df['Age'].median(), inplace=True)
df['City'].fillna(df['City'].mode()[0], inplace=True)
```

7. Adding & Modifying Data

7.1 Create New Columns

```
df['Total'] = df['Maths'] + df['Science']        # sum two columns
df['Average'] = df['Total'] / 2
df['Passed'] = df['Marks'] >= 40                 # boolean column
df['Grade_Upper'] = df['Grade'].str.upper()      # text operation
```

7.2 Apply Custom Functions

```
def classify(marks):
    if marks >= 90: return 'Excellent'
    elif marks >= 70: return 'Good'
    else: return 'Needs Work'
```

```
df['Category'] = df['Marks'].apply(classify)
# OR using lambda (short function):
df['Category'] = df['Marks'].apply(lambda x: 'Pass' if x >= 40 else 'Fail')
```

7.3 Map Values

```
grade_map = {'A': 4.0, 'B': 3.0, 'C': 2.0}
df['GPA'] = df['Grade'].map(grade_map)
```

7.4 Rename, Drop, Replace

```
df.rename(columns={'Name': 'Student', 'Marks': 'Score'}, inplace=True)
df.drop('Grade', axis=1, inplace=True)  # drop column
df.drop(0, axis=0, inplace=True)        # drop row at index 0
df['Grade'].replace('A+', 'A', inplace=True)
```

8. Sorting & Ranking

```
df.sort_values('Marks')                          # ascending
df.sort_values('Marks', ascending=False)         # descending
df.sort_values(['Grade', 'Marks'])                # sort by multiple
df.sort_index()                                  # sort by index
df['Rank'] = df['Marks'].rank(ascending=False)  # rank students
```

9. GroupBy — Summarize by Category

GroupBy splits data into groups and applies a function to each group. Like PIVOT TABLE in Excel.

```
df.groupby('Grade')['Marks'].mean()     # avg marks per grade
df.groupby('Grade')['Marks'].sum()      # total marks per grade
df.groupby('Grade').size()              # count of students per grade
```

```
# Multiple aggregations at once
df.groupby('Grade').agg({
    'Marks': ['mean', 'max', 'min', 'count'],
    'Age': 'mean'
})
```

```
# Custom aggregation
df.groupby('Grade')['Marks'].agg(lambda x: x.max() - x.min())
```

```
# GroupBy with reset_index (makes result a proper DataFrame)
result = df.groupby('City')['Sales'].sum().reset_index()
```

10. Merging & Joining DataFrames

In real projects, data comes from multiple files/tables. You need to join them like SQL.

Join Type	Code	What It Does
Inner join	`pd.merge(df1, df2, on='ID', how='inner')`	Only rows present in BOTH
Left join	`pd.merge(df1, df2, on='ID', how='left')`	All of df1, matching from df2
Right join	`pd.merge(df1, df2, on='ID', how='right')`	All of df2, matching from df1
Outer join	`pd.merge(df1, df2, on='ID', how='outer')`	All rows from BOTH

```
# Stack dataframes vertically (same columns)
combined = pd.concat([df1, df2], ignore_index=True)
# Stack horizontally (same rows)
combined = pd.concat([df1, df2], axis=1)
```

11. String Operations — Text Data

All string operations use .str accessor:

Operation	Code	Example
Lowercase	`df['Name'].str.lower()`	'Alice' → 'alice'
Uppercase	`df['Name'].str.upper()`	'alice' → 'ALICE'
Strip spaces	`df['Name'].str.strip()`	' Alice ' → 'Alice'
Contains	`df['Name'].str.contains('Ali')`	True/False
Starts with	`df['Name'].str.startswith('A')`	True/False

Replace	`df['Name'].str.replace('Mr.', '')`	Remove text
Split	`df['Name'].str.split(' ')`	Split into list
Length	`df['Name'].str.len()`	Number of characters
Slice	`df['Name'].str[0:3]`	First 3 characters
Count pattern	`df['Name'].str.count('a')`	How many 'a' in string

12. Date & Time — Working with Dates

```
# Convert string to datetime
df['Date'] = pd.to_datetime(df['Date'])
df['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')
```

```
# Extract parts of date
df['Year']  = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Day']   = df['Date'].dt.day
df['Weekday'] = df['Date'].dt.day_name()  # 'Monday', etc.
```

```
# Filter by date
df[df['Date'] > '2024-01-01']
df[df['Date'].dt.month == 6]   # only June records
```

13. Pivot Tables & Cross-Tabulation

```
# Pivot table — like Excel pivot
pivot = df.pivot_table(
    values='Sales',
    index='Region',        # rows
    columns='Product',     # columns
    aggfunc='sum',         # how to aggregate
    fill_value=0           # fill missing with 0
)
```

```
# Cross-tabulation (count occurrences)
pd.crosstab(df['Grade'], df['Gender'])
```

14. Changing Data Types

```
df['Marks'] = df['Marks'].astype(int)
df['Price'] = df['Price'].astype(float)
df['Name']  = df['Name'].astype(str)
```

```
df['Category'] = df['Category'].astype('category')  # saves memory!
```

```
# Convert to numeric (errors='coerce' turns non-numbers to NaN)
df['Marks'] = pd.to_numeric(df['Marks'], errors='coerce')
```

15. Common ML Preprocessing Patterns

15.1 Encode Categorical Variables

```
# Label Encoding (manual)
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})
```

```
# One-Hot Encoding — best for ML
df = pd.get_dummies(df, columns=['City', 'Grade'], drop_first=True)
```

15.2 Remove Duplicates

```
df.duplicated().sum()           # count duplicates
df.drop_duplicates(inplace=True)
df.drop_duplicates(subset=['Name'])  # based on specific column
```

15.3 Reset Index

```
df.reset_index(drop=True, inplace=True)
```

15.4 Sample & Shuffle Data

```
df.sample(frac=0.1)             # random 10% of data
df.sample(n=100)                # random 100 rows
df.sample(frac=1, random_state=42).reset_index(drop=True)  # shuffle all
```

15.5 Clip Outliers

```
df['Marks'] = df['Marks'].clip(lower=0, upper=100)
```

16. Quick Reference — Most Used Functions

Category	Function	Purpose
Load	`pd.read_csv()`	Load CSV file
Save	`df.to_csv()`	Save to CSV
Explore	`df.head()/tail()`	See first/last rows
Explore	`df.info()/describe()`	Overview & stats
Select	`df['col']`	Select column
Select	`df.loc[]/df.iloc[]`	Select rows
Filter	`df[df['col'] > x]`	Filter rows
Missing	`df.isnull().sum()`	Count missing
Missing	`df.fillna()`	Fill missing values
Missing	`df.dropna()`	Remove missing rows
Modify	`df['new'] = ...`	Create column
Modify	`df.apply(func)`	Apply function
Modify	`df.rename()`	Rename columns
Group	`df.groupby().agg()`	Group & summarize
Merge	`pd.merge()`	Join two DataFrames
Sort	`df.sort_values()`	Sort by column
Clean	`df.drop_duplicates()`	Remove duplicates
Encode	`pd.get_dummies()`	One-hot encoding
Types	`df.astype()`	Change data type

🎓 **You now know Pandas! Load a real CSV dataset and explore it today.**